

Communication & Core Concepts

How microservices find, talk to, and work with each other

Inter-service Communication

API Gateway

Service Discovery

Data Isolation

API Composition

What We'll Cover

01 Communication Overview

Synchronous vs asynchronous — the two ways services talk

02 The API Gateway

The single entry point — first stop for every request

03 Synchronous Communication

HTTP/REST — services calling each other directly

04 Asynchronous Communication

Message queues — fire and forget, no waiting

05 Service Discovery

How services find each other at runtime — deep dive

06 Data Isolation

Why each service must own its own database

07 API Composition

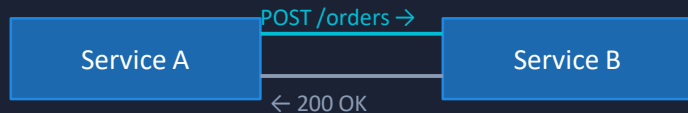
How to query data that lives across multiple services

Two Ways Services Talk

⚡ Synchronous

HTTP / REST

Service A calls Service B and waits for the response before continuing.

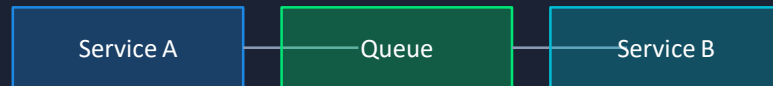


- ✓ Simple to implement
- ✓ Easy to debug & trace
- ✗ Caller is blocked while waiting
- ✗ Tight runtime dependency

✉ Asynchronous

Message Queue (RabbitMQ, Azure Service Bus...)

Service A publishes an event and moves on. Service B picks it up whenever it's ready.



- ✓ Caller doesn't wait (non-blocking)
- ✓ Services are fully decoupled
- ✗ Harder to debug & trace
- ✗ More infrastructure needed

The Single Front Door

Without a gateway, clients need to know the address of every service. The gateway is the only address they need.



🔄 Routing

/products/* → ProductService
/orders/* → OrderService

🔒 Authentication

Validate JWT once here,
not in every service

⚡ Rate Limiting

Block clients sending
too many requests

HTTP Calls Between Services

From our C# store example — OrderService calling ProductService to check stock:

OrderService.cs

```
// Step 1 – register the HTTP client
builder.Services.AddHttpClient("ProductService",
    c => c.BaseAddress = new Uri("http://localhost:5001"));

// Step 2 – call ProductService
var client = httpFactory.CreateClient("ProductService");
var product = await client.GetFromJsonAsync<Product>(
    $
    "/products/{req.ProductId}");
```

1

Register the client

Tell ASP.NET where ProductService lives

2

Inject the factory

IHttpClientFactory creates a configured client

3

Make the HTTP call

GET /products/{id} → deserialize JSON response

4

Handle the result

If null → product doesn't exist → return 400

⚠ What if ProductService is down? → OrderService fails too. This is the risk of synchronous calls.

Events & Message Queues

Scenario: order is placed → notification should be sent. Does OrderService really need to wait for the email?



Event: "OrderPlaced"

```
{
  "orderId": 42,
  "customerEmail": "student@example.com",
  "productName": "Laptop",
  "total": 2599.98
}
```

Why this is better here:

- ✓ OrderService finishes instantly — doesn't wait
- ✓ NotificationService can be down, event is queued
- ✓ Add more consumers (SMS, push) without changing OrderService

The Problem with Hardcoding

In development we hardcode addresses. In production, this breaks everything.

```
c.BaseAddress = new  
Uri("http://localhost:5001");
```

Service Moves

You deploy to a new server or change ports. Every caller has the wrong address hardcoded.

Load Balancing

You spin up 5 instances to handle traffic. Which IP do callers use? They don't know.

Kubernetes / Cloud

Container IPs are dynamic — they change every time a pod restarts. Hardcoding is impossible.

We need a way for services to find each other dynamically — without hardcoded addresses.

How It Works — The Registry

Service Registry

ProductService → 10.0.0.4:5001

OrderService → 10.0.0.7:5002

NotifService → 10.0.0.9:5003

UserService → 10.0.0.2:5004

1

Service registers on startup

When ProductService starts, it tells the registry: "I'm alive at 10.0.0.4:5001"

2

Caller asks the registry

OrderService asks: "Where is ProductService?" → gets back the live address

3

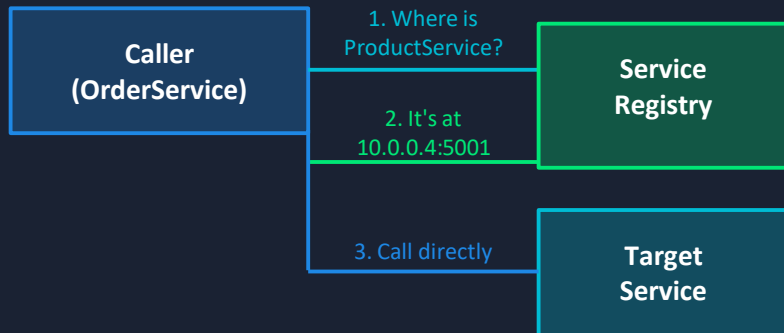
Heartbeat keeps it fresh

Services ping the registry regularly. If they stop, the registry removes them.

Two Discovery Patterns

🔍 Client-Side Discovery

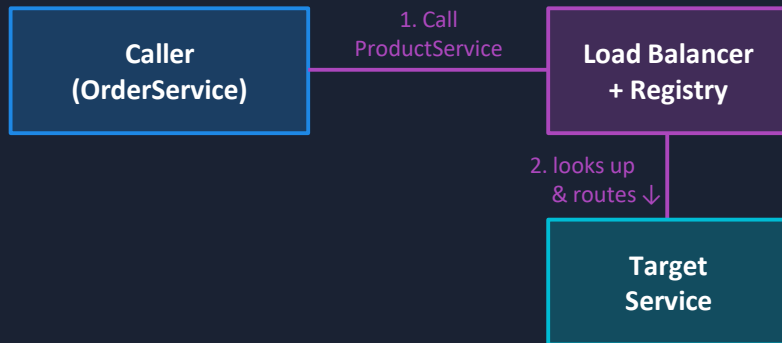
The caller talks directly to the registry, then calls the target service itself.



- ✓ Direct call — no extra hop
- ✓ Can do custom load balancing
- ✗ Every service needs registry code

⚙️ Server-Side Discovery

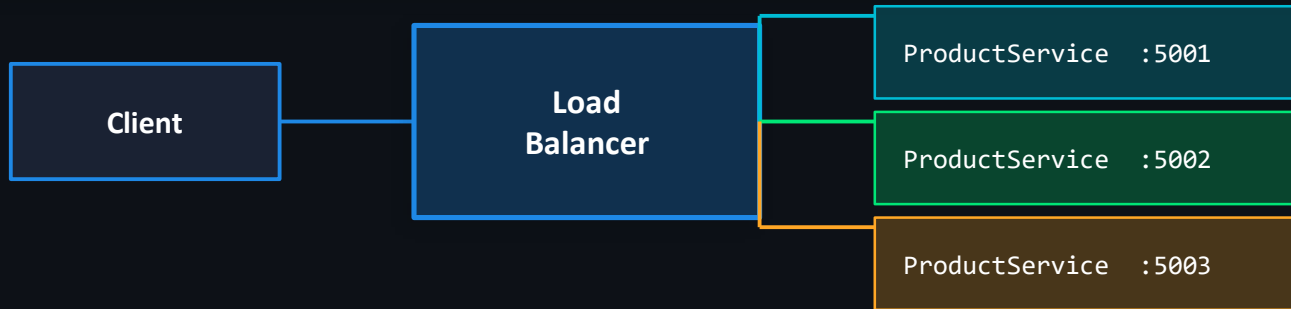
The caller sends the request to a load balancer — it handles lookup and routing.



- ✓ Caller stays simple — no registry code
- ✓ Load balancing handled centrally
- ✗ One extra network hop via balancer

Distributing Traffic Across Instances

When traffic grows you run multiple instances of the same service. The load balancer decides which instance gets each request.



Round Robin

Takes turns: 1→2→3→1→2→3
Simple, works well when all instances have equal power.

Least Connections

Sends to whoever is least busy right now. Better for uneven workloads.

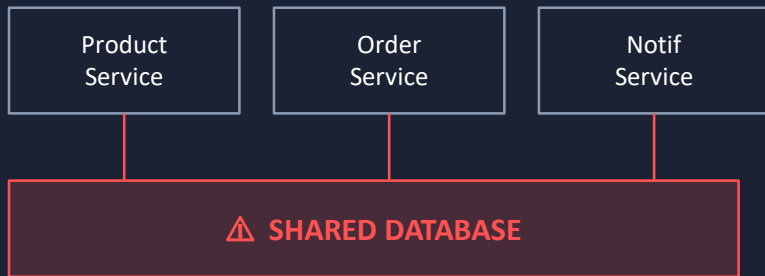
Health Checks

Pings instances regularly. Stops routing to any that don't respond.

Each Service Owns Its Data

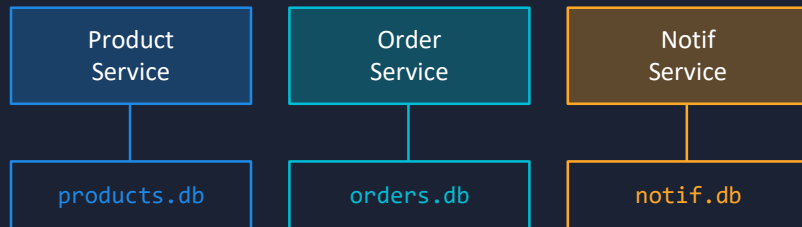
Rule: no two services may share a database or call each other's database directly.

✗ Wrong — Shared DB



- ✗ Schema change breaks all services
- ✗ One slow query affects everyone
- ✗ Can't scale services independently

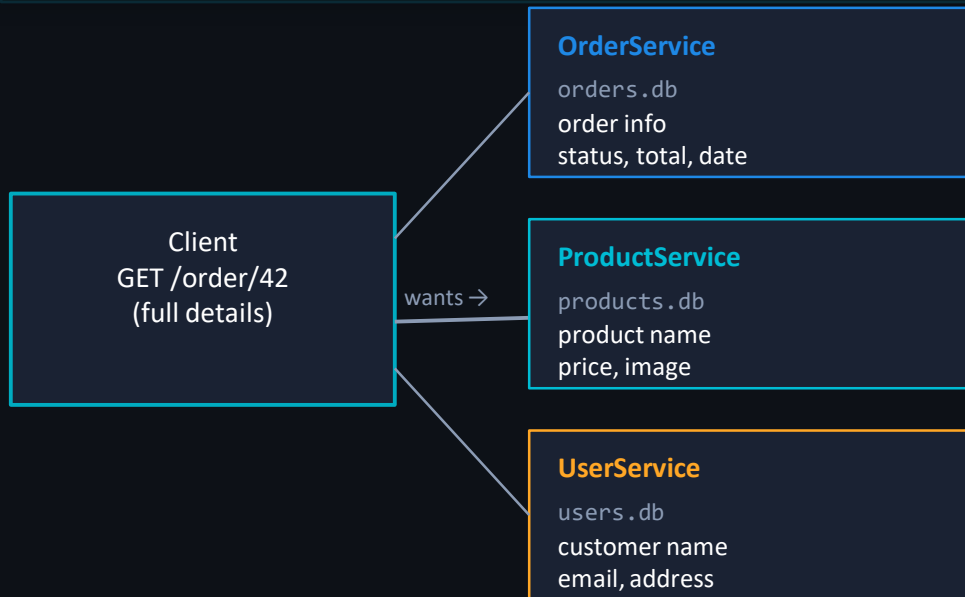
✓ Correct — DB per Service



- ✓ Each service can use a different DB type
- ✓ Schema changes stay isolated
- ✓ Scale storage independently per service

The New Query Problem

Each service owns its own DB — great for isolation. But what if a client needs data from 3 services at once?



✗ Without composition:

- Client makes 3 separate calls
- Client must merge the data itself
- 3x network round-trips
- Client needs to know about all 3 services

✓ With API Composition:

- Client makes 1 call
- Composer fetches and merges
- Client stays simple

Sequential vs Parallel

Sequential

Call each service one after another.
Wait for each before calling the next.

- | | |
|------------------------|---|
| 1. Call OrderService | <code>await GetOrder(42)</code> |
| 2. Call ProductService | <code>await
GetProduct(order.Product
Id)</code> |
| 3. Call UserService | <code>await
GetUser(order.UserId)</code> |
| 4. Merge & return | <code>combine all results</code> |

Total time = sum of all calls

Parallel (Task.WhenAll)

Call all services at the same time.
Merge when all respond.

```
var (order, product, user) =  
    await Task.WhenAll(  
        GetOrder(42),  
        GetProduct(orderId),  
        GetUser(userId)  
    );  
  
return Merge(order, product, user);
```

Total time = slowest single call

⚡ Always prefer parallel — the latency difference is dramatic for 3+ services.

Where It Lives & What Can Go Wrong

API Gateway

Best for simple queries. Gateway calls multiple services and merges before returning. No extra code needed.

Dedicated BFF (Backend for Frontend)

A purpose-built service per client type (mobile, web). More flexibility, handles complex merging.

Client-Side

Client calls services directly. Simple but exposes your service topology and increases client complexity.

What if one service is down?

Fail entire response

Return 503. Clean but shows failure to user.

Partial response

Return what you have, mark missing fields null. Best UX.

Fallback / cache

Return last known data from cache. Good for non-critical data.

→ This failure handling leads naturally into the next chapter: Resilience & Circuit Breakers

Key Takeaways

01

Sync vs Async

HTTP (simple, waits) vs message queues (non-blocking, decoupled). Choose based on whether the caller needs an immediate answer.

02

API Gateway

One entry point. Handles routing, auth, and rate limiting before requests reach any service.

03

Synchronous (HTTP)

Services call each other directly and wait. Simple to debug but creates runtime dependencies.

04

Asynchronous (Events)

Services publish events to a broker. Consumers react at their own pace — fully decoupled.

05

Service Discovery

A registry keeps a live map of addresses. Kubernetes DNS handles this automatically in modern deployments.

06

Data Isolation

Each service owns its own DB. No shared schemas, no cross-service queries — isolation enables independent scaling.

07

API Composition

Query data across services using parallel Task.WhenAll calls. Lives in the Gateway or a BFF service.